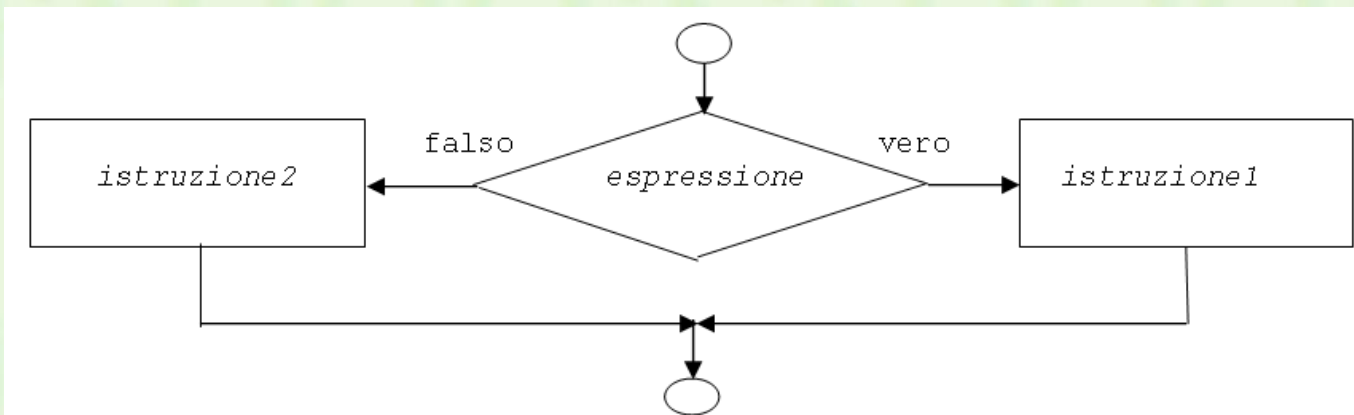
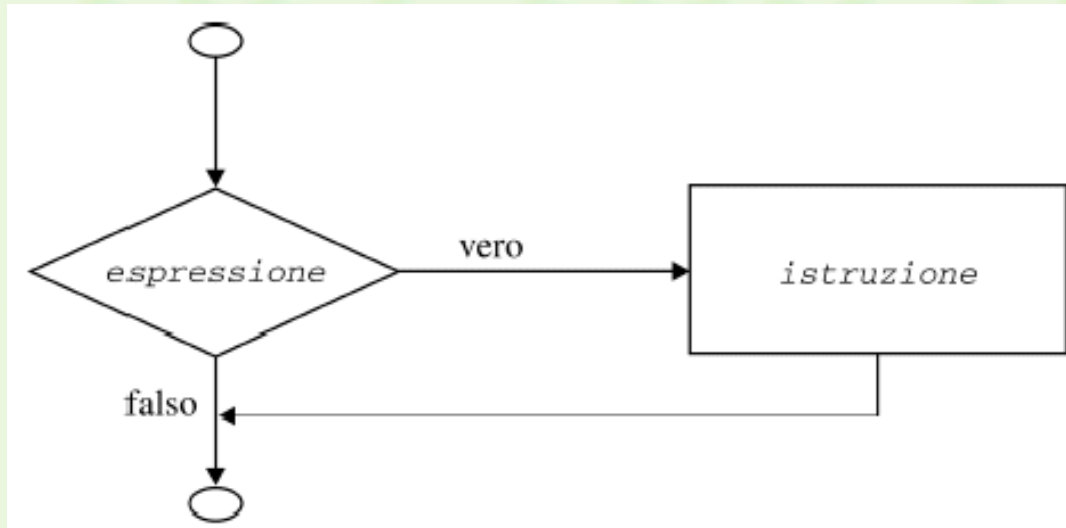


Laboratorio di Programmazione

(Laurea triennale in matematica)

Lezione 6

Due possibili schemi per la selezione:



L'istruzione **if**

- Sintassi

```
if ( <espressione booleana> ){  
    <istruzione_1>;  
    .....  
    <istruzione_r>;  
}  
<istruzione_x>;  
<istruzione_y>;  
.....
```

- Significato

Il blocco *<istruzione_1>..<istruzione_r>* viene eseguito **soltanto** se *<espressione booleana>* è *true*

L'istruzione **if...else**

- Sintassi

```
if ( <espressione booleana> )  
    { <istruzione_1>;  
      .....  
      <istruzione_r>; }  
else  
    { <istruzione_r+1>;  
      .....  
      <istruzione_s>; }  
<istruzione_x>;  
<istruzione_y>;  
.....
```

- Significato

SE <espressione booleana> è **true** viene eseguito Il blocco
<istruzione_1>..**ALTRIMENTI** viene eseguito Il blocco
<istruzione_r+1>..**ALTRIMENTI** viene eseguito Il blocco
<istruzione_r+1>..**ALTRIMENTI** viene eseguito Il blocco
<istruzione_s>;

PROBLEMA 1:

Scrivere un programma che prende in **input 2 numeri interi** inseriti da tastiera e li scrive in **output in ordine crescente** utilizzando in totale soltanto **1 confronto** (con operatore $\leq$) tra i numeri stessi .

No variabili supplementari (solo **2 variabili per i 2 numeri**).

Soluzione: max2.c

```
//max2.c
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int x, y;
```

```
printf ("Scrivi il primo numero  x: ");
```

```
scanf ("%d", &x);
```

```
printf ("Scrivi il secondo  numero y:  ");
```

```
scanf ("%d", &y);
```

```
if (x > y)
```

```
    printf ("I numeri in ordine sono y=%d - x=%d  \n", y,x);
```

```
else
```

```
    printf ("I numeri in ordine sono x=%d - y=%d  \n", x,y);
```

```
return(0);
```

```
}
```

PROBLEMA 2:

Scrivere un programma che prende in **input 3 numeri interi** inseriti da tastiera e calcola il maggiore utilizzando in totale soltanto **2 confronti** (con operatore $\leq$) tra i numeri stessi all'interno di istruzioni if-else annidate.
No variabili supplementari (solo **3 variabili per i 3 numeri**).

Soluzione: max3.c

```
//max3.c
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int num1, num2, num3;
```

```
printf ("Scrivi il primo numero  ");
```

```
scanf ("%d", &num1);
```

```
printf ("Scrivi il secondo  numero  ");
```

```
scanf ("%d", &num2);
```

```
printf ("Scrivi il terzo numero  ");
```

```
scanf ("%d", &num3);
```

<continua>

....

```
if (num1 > num2)
    { if (num1 > num3)
        printf ("Il piu' grande e' il primo! \n\n ");
      else
        printf ("Il piu' grande e' il terzo! \n\n ");
    }
else
    {if (num2 > num3)
        printf ("Il piu' grande e' il secondo! \n\n
");
      else
        printf ("Il piu' grande e' il terzo! \n\n ");
    }
```

PROBLEMA 3:

Scrivere un programma che prende in **input 3 numeri interi** inseriti da tastiera e li scrive in **output in ordine crescente** utilizzando in totale soltanto **3 confronti** (con operatore $\leq$) tra i numeri stessi all'interno di istruzioni if-else annidate.

No variabili supplementari (solo **3 variabili per i 3 numeri**).

Soluzione: (aspetto la vostra!)

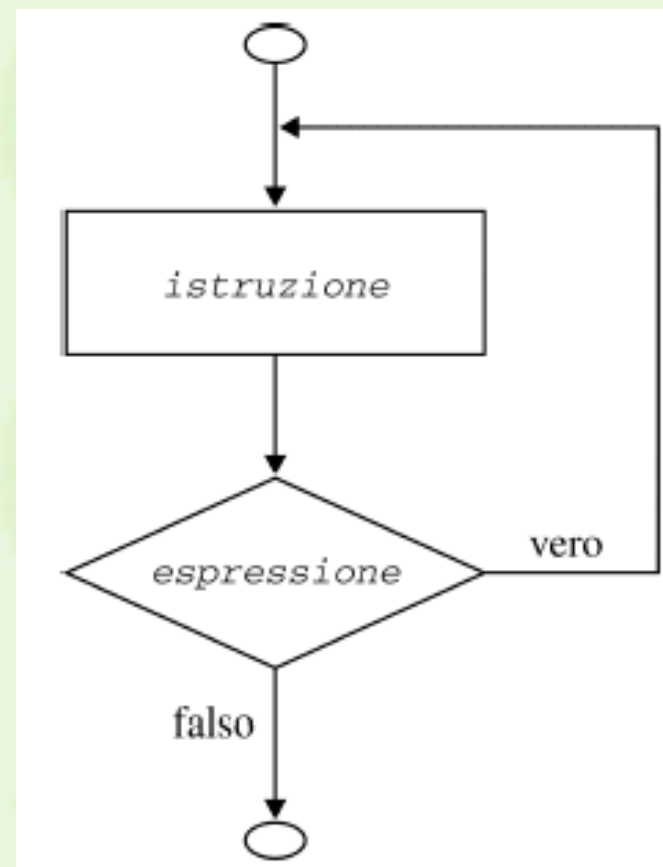
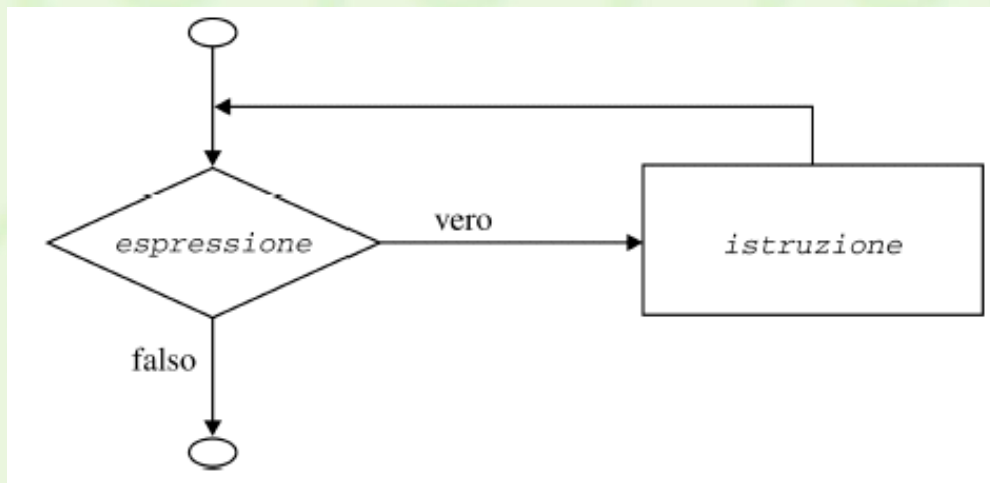
(vedi eseLab)

PROBLEMA 4 (**Importante/Difficile**):

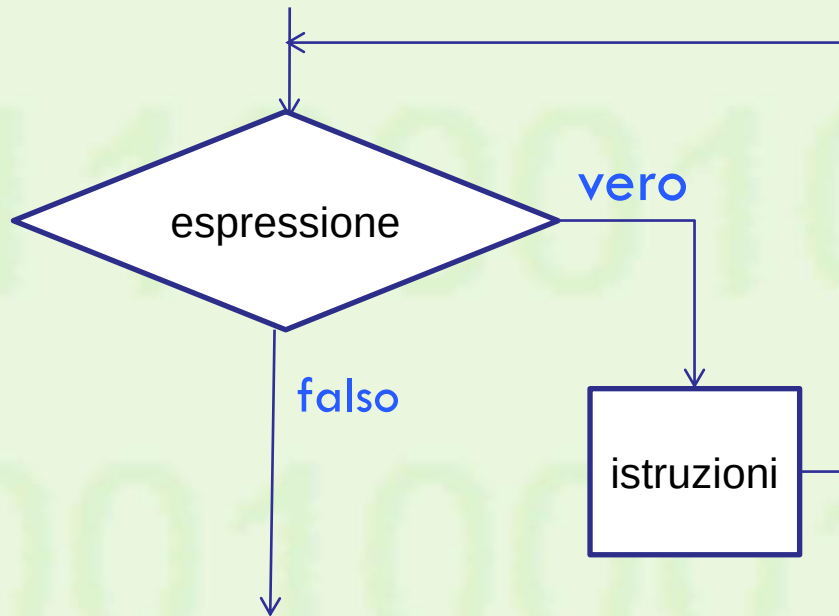
Qual è il **minimo** numero di **operazioni di confronto** necessarie per ordinare in ordine crescente **n numeri** interi?

Lower bound per il problema del sorting.

Due possibili schemi per l'iterazione:



Ciclo **while** (a condizione iniziale)



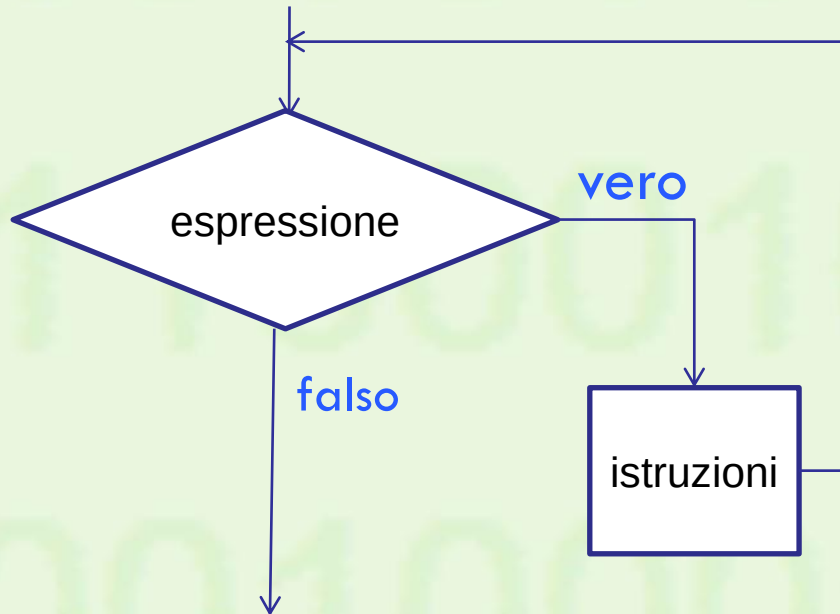
NOTE:

1. è possibile non eseguire mai il blocco
2. le variabili dell'espressione devono essere state inizializzate
3. il blocco deve poter modificare il valore dell'espressione altrimenti verrebbe eseguito indefinitamente (ciclo infinito)
4. Il ciclo termina quando la condizione è falsa

Come funziona:

1. si valuta l'espressione
2. se ha valore vero: *si esegue il blocco di istruzioni e si torna al punto 1*
se ha valore falso: *si passa oltre il ciclo*

Ciclo **while**



```
while (espressione)  
  istruzione;
```

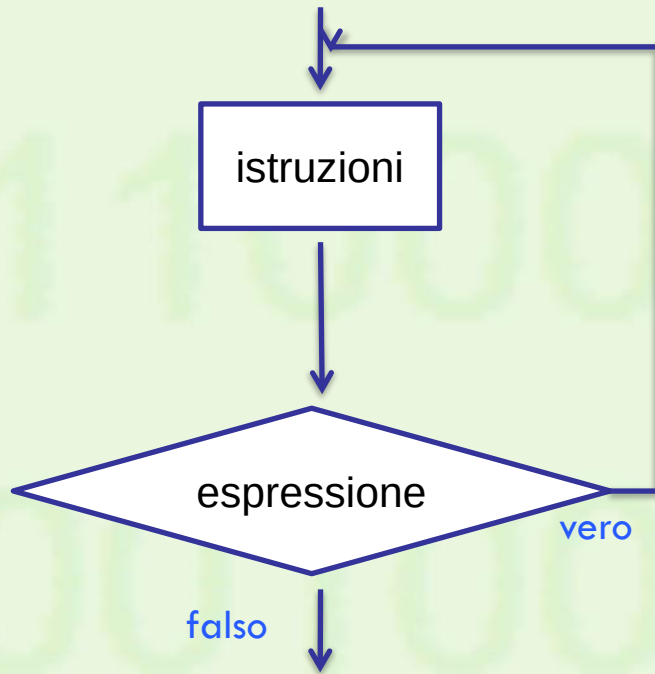
```
while (espressione)  
  {  
    istruzione1;  
    istruzione2;  
  }
```

- Se la condizione ha valore **vero**, esegue il corpo e rivaluta la condizione; altrimenti procede oltre il ciclo.
- Ogni istruzione del blocco termina con “;”

Esempi

- ```
x=1;
while (x<5) {
 printf("ciao!\n");
 x=x+1;
}
```
- ```
x=1; // CICLO INFINITO
while (x<5){
    printf("ciao!\n");
}
```
- ```
while (1) { // CICLO INFINITO
 printf("ciao!\n");
}
```
- ```
x=10; //CICLO NULLO
while (x<5){
    printf("ciao!\n");
}
```

Ciclo **do...while** (a condizione finale)



NOTE:

1. il blocco si esegue almeno una volta
2. le variabili dell'espressione devono essere state inizializzate
3. Se l'espressione è vera, il blocco deve poter modificare il valore dell'espressione altrimenti verrebbe eseguito indefinitamente (ciclo infinito)
4. Si esce dal ciclo quando la condizione risulta falsa

Come funziona:

1. si esegue il blocco
2. si valuta l'espressione e se ha valore vero si torna al punto 1, se ha valore falso: si passa oltre il ciclo

Ciclo do...while



```
do istruzione while  
(espressione) ;
```

```
do  
  {  
    istruzione1;  
    istruzione2;  
  }  
while (espressione) ;
```

- Esegue il corpo una volta; se la condizione ha valore vero, lo riesegue e rivaluta la condizione; altrimenti procede oltre il ciclo
- ogni istruzione termina con ;
- l'intero costrutto termina con ;

Istruzione `do..while`

- Sintassi

```
do{  
    <istruzione_1>  
    ...  
    <istruzione_r>  
}  
while (<espressione booleana>  
    <istruzione_x>;  
    <istruzione_y>;  
    .....
```

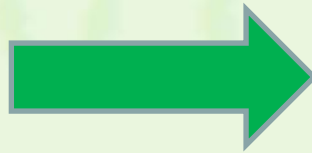
- Significato

Il blocco `<istruzione_1>..<istruzione_r>` viene eseguito **fintanto che** `<espressione booleana>` è *true* (viene eseguito **almeno una volta**).

Equivalenza **while** – **do...while**

- Dato un ciclo **do...while** si può scrivere il ciclo **while** equivalente

```
do  
{  
  istruzioni  
}  
while (espressione) ;
```

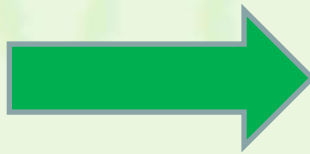


```
{  
  istruzioni  
}  
while (espressione)  
{  
  istruzioni  
}
```

Equivalenza **while** – **do...while**

- I cicli a condizione iniziale e a condizione finale sono equivalenti. Per esempio, dato un ciclo **while** si può scrivere il ciclo **do...while** equivalente

```
while  
  (espressione)  
{  
  istruzioni  
}
```



```
if (espressione)  
{  
  do  
  {  
    istruzioni  
  }  
  while (espressione);  
}
```

```
while (not edge) {  
  run();  
}
```

```
do {  
  run();  
} while (not edge);
```



Operatori incremento/decremento

- Gli operatori incremento `++` e decremento `--` sono operatori unari (serve un unico operando): aggiungono (tolgono) l'unità (1) all'operando.
- Equivalgono ad assegnare alla stessa variabile il proprio valore incrementato
- **`x++` è lo stesso di `x=x+1`**
- **`x--` è lo stesso di `x=x-1`**
- ATTENZIONE: due diverse modalità di esecuzione, qualora l'incremento sia parte di un'istruzione più complessa.
- **`x++` ;** prima usa `x` nell'istruzione, poi lo incrementa
- **`++x` ;** prima incrementa `x`, poi lo usa nell'istruzione

Operatori incremento decremento

ATTENZIONE!

Le due istruzioni di incremento (decremento) *isolate* hanno lo stesso effetto. Il risultato cambia quando vengono usate in istruzioni meno semplici.

```
int n, m=0;  
n = m++;
```

n vale 0, m vale 1

```
int n, m=0;  
n = ++m;
```

n vale 1, m vale 1

Incrementi e decrementi

L'incremento di un valore diverso da uno

lo si ottiene con +=

```
somma = somma+9;
```

```
somma += 9;
```

La forma generalizzata è

variabile [operatore]= espressione

dove ***operatore*** può essere **+** **-** ***** **/** **%**

La si utilizza quando la variabile appare a sinistra e a destra di un assegnamento.

Forma compatta

```
a *= 5;
```

```
a -= b;
```

```
a *= 4+b;
```

Forma classica

```
a = a*5;
```

```
a = a-b;
```

```
a = a*(4+b);
```

Regole di precedenza

Più alta

- Operatori *unari* $+$, $-$, $!$, $++$, $--$
- Operatori aritmetici *binari* $*$, $/$, $\%$
- Operatori aritmetici binari $+$, $-$
- Operatori relazionali $<$, $>$, $<=$, $>=$
- Operatori di uguaglianza $==$, $!=$
- Operatore booleano $\&\&$
- Operatore booleano $||$
- Operatori assegnamento $=$, $+=$, $-=$, $*=$, $/=$

Più bassa

Il costrutto **for**

E' un costrutto di iterazione come il while e il do while.

Equivalente agli altri due!

Si usa nei casi di **iterazione definita da contatore**

Istruzione **for**

- Sintassi

```
for (<espr1> ; <espr2> ; <espr3> )  
{ <istruzione_1>  
    ...  
    <istruzione_r>  
}  
<istruzione_x>;  
.....
```

Come funziona:

- valuta/ esegue *<espr1>* [inizializzazione]
- valuta (o esegue) *<espr2>*: se ha valore falso procede oltre il ciclo; se ha valore vero, esegue il corpo e poi valuta (o esegue) *<espr3>* [modifica valori di <espr2>]

Esempi

```
c=1;  
while (c<=n){  
    printf(" * \n");  
    c++;  
}
```

```
for(c=1; c<=n; c++)  
    printf("*\n");
```

Esempi

```
c=0;  
while (c<=n) {  
    c++;  
    printf( ' '%d' ',c) ;  
}
```

```
for(c=0; c<=n; c++)  
    printf( ' '%d\n' ', c+1) ;
```

Esempi di cicli di conteggi

Conteggio da 0 a n:

```
for (i=0;i<n;i++)  
{  
    istruzioni;  
}
```

Conteggio da n a 0:

```
for (i=n;i>0;i--)  
{  
    istruzioni  
}
```

Conteggio da 1 a n:

```
for (i=1;i<=n;i+=1)  
{  
    istruzioni  
}
```

Conteggio da n a 1:

```
for (i=n;i>=1;i=i-1)  
{  
    istruzioni  
}
```

Equivalenza **for** - **while**

- I cicli **for** equivalgono ai cicli **while**

```
for (espr1 ; espr2 ; espr3)  
{  
    istruzioni  
}
```



```
espr1;  
while (espr2)  
{  
    istruzioni  
    espr3;  
}
```